REQUEST FOR COMMENTS

RFC — Proposta de Projeto de Portfólio

Engenharia de Software — Católica SC

InfraObserver

Sistema de Monitoramento Centralizado de Endpoints em Rede

Título do Projeto	InfraObserver — Sistema de Monitoramento Centralizado de Endpoints
Linha de Projeto	Web App / Ferramenta de Infraestrutura e Monitoramento
Autor	Bryann Lucas Locatelli
Data da Proposta	Abril / 2026
Versão	1.0
Disciplina / Curso	PAC VII — Engenharia de Software
Repositório	https://github.com/LocatelliBryann/InfraObserver

1. Visão do Produto e Impacto

1.1 Contexto e Problema

Organizações que possuem múltiplos computadores conectados em rede, como pequenas empresas, escritórios, cartórios e instituições de ensino, enfrentam dificuldades para acompanhar de forma centralizada o estado dos seus endpoints. A ausência de uma visão consolidada sobre desempenho, disponibilidade e atividades relevantes dos computadores dificulta a identificação precoce de problemas e aumenta o tempo necessário para diagnóstico e suporte. Esse problema afeta diretamente dois grupos distintos:

- O administrador de rede ou de TI, que precisa identificar manualmente máquinas com alto consumo de CPU, memória insuficiente, espaço em disco crítico, utilização anormal de recursos ou simplesmente indisponíveis. Sem uma ferramenta centralizada, esse processo depende de acesso físico ou remoto individual a cada máquina, tornando-o lento, reativo e propenso a falhas não detectadas a tempo.
- O gestor ou responsável técnico da instituição, que não possui visibilidade operacional do parque de máquinas sob sua responsabilidade, dificultando decisões relacionadas à manutenção preventiva, substituição de hardware ou investigação de incidentes.

Embora existam soluções de monitoramento no mercado, muitas delas são voltadas para ambientes corporativos maiores, exigindo infraestrutura dedicada, processos de implantação mais complexos ou conhecimentos especializados. Em organizações menores, é comum que o acompanhamento dos endpoints ainda seja realizado de forma manual e reativa.

Além do monitoramento de desempenho, existe a necessidade de registrar eventos relevantes nos sistemas de arquivos dos endpoints, como criação, modificação, exclusão ou renomeação de arquivos em diretórios monitorados. Essas informações podem auxiliar atividades de auditoria, rastreabilidade e suporte operacional.

Nesse contexto, surge a oportunidade de desenvolver uma solução leve e centralizada que permita monitorar endpoints, acompanhar métricas em tempo quase real, detectar indisponibilidades e registrar eventos relevantes de forma simplificada, sem a necessidade de infraestrutura complexa.

1.2 Origem da Demanda e Evidências

Contexto Acadêmico e Prático

A demanda surgiu da observação de ambientes com múltiplos computadores conectados em rede, incluindo instituições de ensino, pequenas empresas e ambientes corporativos de pequeno porte. Nesses cenários, o acompanhamento do estado dos endpoints geralmente ocorre de forma reativa, após relatos de lentidão, indisponibilidade ou falhas percebidas pelos usuários.

Pesquisa com Usuários

Foram realizadas conversas informais com profissionais de TI e usuários responsáveis pela administração de ambientes computacionais. As principais dificuldades identificadas foram:

- Dependência de verificações manuais para identificar problemas de desempenho em computadores da rede;
- Baixa visibilidade sobre o consumo de recursos dos endpoints antes da ocorrência de chamados;
- Dificuldade em identificar rapidamente máquinas indisponíveis ou com comportamento anormal;
- Necessidade de uma solução centralizada, simples de implantar e executada na infraestrutura local da organização;
- Interesse em acompanhar eventos relevantes de sistema de arquivos para fins de auditoria e suporte operacional.

As observações realizadas indicam a oportunidade de uma solução que centralize métricas de desempenho, eventos de sistema de arquivos e informações de disponibilidade dos endpoints em uma única interface, reduzindo o tempo necessário para diagnóstico e acompanhamento do ambiente.

Essas conversas foram informais e não formam uma amostra estatística: indicam a percepção dos profissionais consultados, não uma medição. Por isso, a validação do produto com usuários reais (piloto em ambiente de produção, teste de usabilidade e testes de desempenho) faz parte do planejamento e está descrita na seção 7.1.

1.3 Análise de Soluções Existentes (Benchmark)

Foram analisadas as principais soluções existentes no mercado, apresentadas na tabela 01:

Solução	Público-Alvo	Pontos Fortes	Limitações
Prometheus + Grafana	Equipes DevOps e SRE corporativas	Altamente escalável, ecossistema maduro, dashboards avançados	Exige configuração inicial de múltiplos componentes e maior conhecimento técnico para implantação e manutenção.
Zabbix	Médias e grandes empresas	Agente leve, alertas configuráveis, suporte SNMP	Implantação mais abrangente e com maior número de configurações para ambientes pequenos.
Datadog / New Relic	Empresas enterprise	Observabilidade completa, APM integrado, fácil setup	Modelo SaaS pago, inviável para organizações com recursos limitados ou pequenas organizações
Netdata	Sysadmins individuais	Leve, real-time, fácil de instalar	Foco em máquina individual, multi-host exige plano pago, sem monitoramento de eventos de arquivos

Tabela 01: Principais soluções existentes no mercado.

Diferencial do Projeto InfraObserver

O InfraObserver não busca substituir soluções consolidadas de monitoramento, mas oferecer uma alternativa simplificada para ambientes de pequeno e médio porte que necessitam de monitoramento básico de endpoints, implantação simplificada e controle local dos dados. O InfraObserver se diferencia em três pontos centrais:

- Comunicação baseada em agentes que enviam métricas e eventos diretamente ao servidor central via HTTP/JSON, reduzindo a necessidade de configurações adicionais na infraestrutura monitorada. Custo zero e hardware modesto — desenvolvido do zero, sem dependência de serviços pagos, executável em servidores simples ou até em uma máquina local.
- Desenvolvido utilizando tecnologias open source e executável em infraestrutura local de baixo custo.
- Integra monitoramento de desempenho e rastreamento de eventos de sistema de arquivos em uma única solução, permitindo registrar criação, modificação, exclusão e renomeação de arquivos em diretórios monitorados.
- Operação totalmente local (on-premises), permitindo que organizações mantenham métricas, eventos e configurações sob seu próprio controle, sem dependência de serviços externos ou infraestrutura em nuvem.

O InfraObserver diferencia-se por combinar monitoramento de desempenho, detecção de indisponibilidade e rastreamento de eventos de sistema de arquivos em uma solução leve, executada localmente e voltada para ambientes que necessitam de implantação simplificada e baixo custo operacional.

1.4 Público-Alvo

Foi realizada uma análise do público-alvo da aplicação, cujos perfis são apresentados na Tabela 02:

Perfil	Descrição
Administrador de TI / Suporte Técnico	Profissional responsável pela administração e manutenção dos computadores da organização. Utiliza o dashboard para acompanhar métricas, identificar problemas de desempenho, monitorar endpoints e investigar alertas. Busca uma solução simples de implantar e operar.
Gestor / Coordenador de TI	Responsável pelo acompanhamento operacional do ambiente computacional. Utiliza o dashboard para obter uma visão consolidada dos endpoints, acompanhar disponibilidade dos equipamentos e apoiar decisões relacionadas à manutenção e infraestrutura.
Pequenas e Médias Organizações	Empresas, escritórios, cartórios, instituições de ensino e outros ambientes que necessitam monitorar seus computadores de forma centralizada, sem depender de soluções complexas ou infraestrutura dedicada.

Tabela 02: Público-alvo da aplicação.

1.5 Objetivos do Projeto

Objetivo Geral

Desenvolver o InfraObserver, um sistema de monitoramento centralizado de endpoints composto por um agente leve, uma API REST e um dashboard web, capaz de coletar métricas de desempenho, registrar

eventos de sistema de arquivos, detectar endpoints inativos e disponibilizar informações em tempo quase real para administradores de TI.

Objetivos Específicos

- Desenvolver um agente leve em Python para Windows, responsável por coletar métricas de CPU, memória, disco e rede, além de detectar eventos de criação, modificação, exclusão e renomeação de arquivos em diretórios monitorados.
- Implementar uma API REST utilizando Node.js e Express para autenticar agentes, receber métricas e eventos, persistir informações em banco PostgreSQL e disponibilizar os dados para consulta pelo dashboard.
- Desenvolver um dashboard web em React para visualização centralizada de endpoints, métricas em tempo quase real, histórico de desempenho, eventos de sistema de arquivos e alertas gerados pelo sistema.
- Implementar mecanismos automáticos de detecção de endpoints inativos e geração de alertas quando métricas monitoradas ultrapassarem limites configurados.
- Permitir o gerenciamento das configurações dos agentes, incluindo diretórios monitorados, intervalo de coleta e controle de versão.
- Aplicar conceitos de arquitetura cliente-servidor, APIs REST, persistência de dados, observabilidade, containers Docker e integração contínua no desenvolvimento da solução.

1.6 Métricas de Sucesso (KPIs)

Os principais indicadores de desempenho (KPIs) estabelecidos para o projeto são apresentados na Tabela 03.

Indicador	Meta
Tempo de resposta da API REST	95% das requisições respondidas em menos de 300 ms
Latência de detecção de endpoint inativo	Endpoint marcado como OFFLINE após ausência de dados por 3 ciclos consecutivos de coleta
Suporte a endpoints simultâneos	Mínimo de 50 endpoints ativos sem degradação (porte do ambiente piloto, com 45 estações)
Compatibilidade de SO do agente	Compatibilidade com Windows 10 e 11; suporte Linux previsto para versões futuras
Overhead de CPU do agente no endpoint	Inferior a 2% de CPU média durante coleta
Detecção de eventos de sistema de arquivos	Registro de eventos em até 10 segundos após a ocorrência, incluindo a espera pela auditoria do Windows
Disponibilidade do Servidor Central	99% de disponibilidade durante os testes do

Indicador	Meta
	projeto
Persistência de métricas	100% das métricas recebidas devem ser armazenadas com sucesso no banco de dados
Atualização do Dashboard	Atualização automática das informações em até 30 segundos

Tabela 03: Métricas de sucesso definidas para o projeto.

Na versão 1.1, as metas foram ajustadas ao porte do ambiente piloto e cada uma passou a ter uma forma de medição definida, apresentada na seção 7.1.

2. Engenharia de Requisitos

2.1 Personas

Persona 1 — Técnico de TI de Cartório

Carlos, 21 anos, Técnico de TI em um cartório de registro de imóveis.

Responsável por aproximadamente 50 estações de trabalho e alguns servidores internos. Atualmente monitora os computadores de forma manual, acessando remotamente as máquinas quando usuários relatam lentidão, falhas ou indisponibilidade. Tem dificuldade em identificar rapidamente equipamentos com uso excessivo de recursos ou computadores que deixaram de responder na rede. Busca uma solução simples, centralizada e executada localmente, sem dependência de serviços em nuvem, permitindo visualizar o estado dos endpoints e eventos importantes em um único painel.

Persona 2 — Analista de Suporte em Pequena Empresa

Marina, 29 anos, analista de suporte em uma empresa de contabilidade com 15 computadores.

Presta suporte presencial e remoto aos colaboradores. Possui conhecimento técnico intermediário e utiliza ferramentas de acesso remoto diariamente. Sente falta de visibilidade sobre o estado dos computadores antes da abertura de chamados. Deseja acompanhar métricas de CPU, memória, disco e eventos de arquivos em tempo quase real, permitindo identificar problemas de forma proativa e reduzir o tempo gasto em diagnósticos.

Persona 3 — Administrador de Infraestrutura

Ricardo, 37 anos, Administrador de Infraestrutura em uma empresa com múltiplos setores.

Responsável por garantir a disponibilidade dos computadores utilizados pelos colaboradores. Necessita identificar rapidamente endpoints offline, acompanhar tendências de consumo de recursos e receber alertas quando métricas ultrapassarem limites configurados. Prefere soluções instaladas na infraestrutura da própria organização, mantendo total controle sobre os dados monitorados.

2.2 Requisitos Funcionais (RF)

Os requisitos funcionais definidos para o desenvolvimento do InfraObserver são apresentados na Tabela 04. Esses requisitos descrevem as funcionalidades que o sistema deverá oferecer para atender aos objetivos do projeto.

ID	Requisito	Prioridade
RF01	O sistema deve permitir que o agente envie métricas de CPU, memória, disco e rede ao servidor central via HTTP/JSON	**Alta**
RF02	O sistema deve permitir que o agente detecte e envie eventos de criação, modificação, exclusão e renomeação de arquivos em diretórios	**Alta**

ID	Requisito	Prioridade
	monitorados.	
RF03	O sistema deve receber, validar e persistir as métricas e eventos enviados pelos agentes	**Alta**
RF04	O sistema deve detectar automaticamente endpoints inativos com base na ausência de sinal dentro de um intervalo configurável	**Alta**
RF05	O sistema deve expor uma API REST para que o dashboard consulte endpoints, métricas e eventos	**Alta**
RF06	O sistema deve permitir que o dashboard liste todos os endpoints cadastrados e seus status (online / offline / alerta)	**Alta**
RF07	O sistema deve exibir as métricas de um endpoint em tempo quase-real no dashboard	**Alta**
RF08	O sistema deve apresentar histórico de métricas em gráficos de série temporal no dashboard	Média
RF09	O sistema deve registrar e exibir os eventos de sistema de arquivos detectados pelo agente	Média
RF10	O sistema deve permitir que o administrador configure remotamente os diretórios monitorados por cada agente através do dashboard.	Média
RF11	O sistema deve emitir alertas visuais no dashboard quando uma métrica ultrapassar um limite pré-definido (threshold) ou quando um endpoint ficar offline.	**Alta**
RF12	O sistema deve registrar o histórico de alertas gerados por endpoint	Baixa
RF13	O sistema deve registrar automaticamente novos endpoints quando um agente válido realizar sua primeira conexão com o servidor central.	**Alta**
RF14	O sistema deve autenticar cada agente com uma credencial própria, obtida na instalação a partir do token de instalação do cliente, antes de aceitar métricas ou eventos enviados.	**Alta**
RF15	O sistema deve registrar o usuário responsável por cada evento de sistema de arquivos, obtido da auditoria do Windows quando disponível ou, na falta dela, o usuário com sessão aberta no endpoint.	Média
RF16	O sistema deve atualizar automaticamente o status de um endpoint para ONLINE quando novas métricas forem recebidas após um período de inatividade.	**Alta**
RF17	O sistema deve permitir configurar o intervalo de coleta de métricas realizado pelos agentes.	Média
RF18	O sistema deve permitir consultar métricas históricas filtradas por período.	Média
RF19	O agente deve verificar periodicamente a existência de novas versões	Baixa

ID	Requisito	Prioridade
	disponibilizadas pelo servidor.	
RF20	O sistema deve permitir consultar informações detalhadas de um endpoint, incluindo métricas atuais, histórico e eventos registrados.	Alta
RF21	O sistema deve exigir autenticação do administrador (e-mail e senha) para acessar o dashboard e as consultas da API.	Alta
RF22	O sistema deve permitir que o administrador gere e revogue tokens de instalação por cliente e revogue o acesso de um endpoint específico.	Alta
RF23	O sistema deve remover automaticamente métricas, eventos de sistema de arquivos e alertas mais antigos que o período de retenção definido.	Alta

Tabela 04: Requisitos funcionais do sistema InfraObserver.

2.3 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definidos para o InfraObserver são apresentados na Tabela 05. Esses requisitos estabelecem critérios de qualidade, desempenho, segurança, compatibilidade e demais características necessárias para o correto funcionamento do sistema.

ID	Requisito	Categoria
RNF01	O agente deve consumir no máximo 2% de CPU em média durante os ciclos de coleta	Desempenho
RNF02	A API deve fornecer tempo de resposta compatível com atualização em tempo quase real das métricas monitoradas.	Desempenho
RNF03	O agente deve ser compatível com Windows 10/11 (obrigatório) e Linux Ubuntu 22.04+ (desejável)	Compatibilidade
RNF04	O sistema deve suportar no mínimo 50 endpoints ativos simultaneamente sem degradação perceptível de desempenho.	Escalabilidade
RNF05	A comunicação entre agente e servidor deve utilizar HTTPS e autenticação por credencial individual de cada endpoint, para evitar envio não autorizado de dados.	Segurança
RNF06	O dashboard deve ser responsivo e funcional em navegadores modernos (Chrome, Firefox, Edge)	Usabilidade
RNF07	O sistema deve permitir configuração do intervalo de coleta (candidato: 60 segundos)	Configurabilidade
RNF08	Métricas, eventos de sistema de arquivos e alertas devem ser retidos por 60 dias (valor padrão, configurável) e removidos automaticamente após esse prazo	Armazenamento
RNF09	O servidor central deve ser executável em ambiente local (on-premises), sem dependência obrigatória de serviços em nuvem.	Portabilidade

ID	Requisito	Categoria
RNF10	O sistema deve iniciar automaticamente após reinicialização do servidor central.	Confiabilidade
RNF11	O agente deve operar como serviço do sistema operacional, iniciando automaticamente junto ao Windows.	Operacionalidade
RNF12	Os componentes do servidor central devem ser executáveis através de containers Docker.	Implantação
RNF13	As métricas e eventos armazenados não devem ser perdidos em caso de reinicialização do servidor.	Confiabilidade
RNF14	O dashboard deve atualizar automaticamente os dados sem necessidade de recarregamento manual da página.	Usabilidade
RNF15	Senhas e tokens devem ser armazenados apenas como hash (bcrypt para senhas e SHA-256 para tokens) e nunca registrados em logs.	Segurança

Tabela 05: Requisitos não funcionais do sistema InfraObserver.

2.4 Casos de Uso Principais

Os principais casos de uso identificados para o sistema InfraObserver são apresentados na Tabela 06. Esses casos descrevem as principais interações entre os atores do sistema e as funcionalidades disponibilizadas pela aplicação.

ID	Nome	Ator	Descrição
UC01	**Registrar endpoint**	Agente	O agente realiza o primeiro envio de dados ao servidor, que cadastra automaticamente o endpoint com um identificador único (hostname + IP).
UC02	**Enviar métricas**	Agente	Em intervalos regulares, o agente coleta métricas do sistema e as envia ao servidor central via HTTP POST. O servidor válida e persiste os dados.
UC03	**Enviar evento de arquivo**	Agente	Ao detectar criação, modificação, exclusão ou renomeação de arquivos em diretórios monitorados, o agente envia o evento ao servidor contendo timestamp, caminho do arquivo e usuário logado.
UC04	**Visualizar painel geral**	Administrador	Administrador acessa o dashboard e visualiza a lista de todos os endpoints com status (online/offline/alerta), última coleta e métricas resumidas.
UC05	**Detalhar endpoint**	Administrador	Administrador seleciona um endpoint específico e visualiza métricas atuais, histórico de métricas, eventos registrados e alertas associados ao endpoint.
UC06	**Visualizar eventos**	Administrador	Administrador consulta o registro de eventos

ID	Nome	Ator	Descrição
	de arquivo		detectados nos diretórios monitorados de um endpoint, com filtros por data.
UC07	**Receber alerta visual**	Administrador	Ao acessar o dashboard, o administrador visualiza alertas gerados por métricas acima dos limites configurados ou por endpoints marcados como offline.
UC08	**Autenticar agente**	Agente	Na primeira conexão, o agente troca o token de instalação do cliente por uma credencial própria. Depois, envia essa credencial em cada requisição para validação pelo servidor central.
UC09	**Configurar diretórios monitorados**	Administrador	O administrador define quais diretórios serão monitorados por cada agente para detecção de eventos de sistema de arquivos.
UC10	**Configurar intervalo de coleta**	Administrador	O administrador define o intervalo de coleta de métricas utilizado pelos agentes monitorados.

Tabela 06: Casos de uso principais do sistema.

Os principais casos de uso identificados para o sistema InfraObserver são representados na Figura 01. O diagrama apresenta os atores envolvidos e suas respectivas interações com suas funcionalidades.

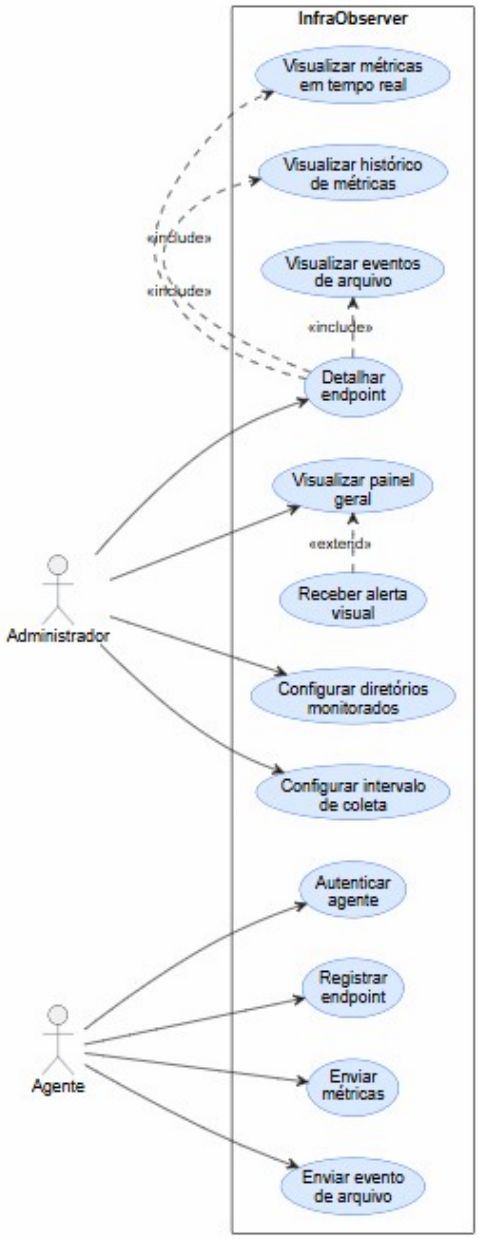


Figura 01: Diagrama de casos de uso do sistema InfraObserver.

2.5 Regras de Negócio

As principais regras de negócio definidas para o funcionamento do InfraObserver são apresentadas na Tabela 07. Essas regras estabelecem restrições e comportamentos que devem ser respeitados pelo sistema durante sua operação.

ID	Regra
RN01	Cada endpoint deve possuir um identificador único gerado durante o registro inicial do agente. Hostname e endereço IP são utilizados apenas como informações de identificação complementar.

ID	Regra
RN02	Um endpoint é considerado inativo (offline) quando não envia dados ao servidor por um período superior a 3 ciclos consecutivos do intervalo de coleta configurado.
RN03	Um endpoint entra em estado de alerta quando ao menos uma métrica monitorada ultrapassa o threshold definido. O estado de alerta persiste até que a métrica retorne ao nível normal em pelo menos 2 coletas consecutivas.
RN04	O agente só pode enviar dados ao servidor com uma credencial válida e não revogada, que vale apenas para o seu próprio endpoint. Requisições sem autenticação válida são rejeitadas.
RN05	Cada agente deve utilizar o intervalo de coleta definido em sua configuração recebida do servidor.
RN06	Métricas, eventos de sistema de arquivos e alertas são mantidos por 60 dias (valor padrão, configurável pela organização). Uma rotina diária remove automaticamente os registros mais antigos.
RN07	Os diretórios monitorados são definidos na instalação do agente. A configuração remota pelo dashboard (RF10) é uma evolução prevista e não faz parte do MVP.
RN08	Eventos de sistema de arquivos devem ser associados ao usuário que realizou a ação, segundo a auditoria do Windows. Sem esse registro, é usado o usuário com sessão aberta, e o evento indica qual das duas origens foi usada.
RN09	Um endpoint previamente marcado como OFFLINE retorna automaticamente ao estado ONLINE ao enviar uma nova coleta válida ao servidor.
RN10	O primeiro envio válido realizado por um agente autenticado deve registrar automaticamente o endpoint no servidor.
RN11	Um endpoint com acesso revogado tem suas requisições recusadas. O histórico já registrado permanece até o fim do período de retenção.
RN12	Um token de instalação só gera credenciais para computadores do cliente para o qual foi criado e deixa de funcionar quando revogado ou vencido.

Tabela 07: Regras de negócio do sistema InfraObserver.

2.6 Fora do Escopo

O sistema não contempla, nesta versão, as seguintes funcionalidades:

- Envio de notificações externas ao dashboard (e-mail, SMS, webhook) — alertas são exibidos apenas na interface web.
- Monitoramento de processos individuais em execução nos endpoints.
- Gerenciamento remoto dos endpoints a partir do dashboard (reinicialização, execução de comandos).
- Suporte a macOS como sistema operacional do agente.
- Integração com sistemas externos de monitoramento, SIEM, SNMP ou plataformas de observabilidade.
- Inventário de softwares instalados nos endpoints.
- Inventário detalhado de hardware dos endpoints.

- Login de clientes finais, para cada organização ver apenas os próprios computadores (o dashboard do administrador já permite filtrar por cliente).
- Autenticação em dois fatores (MFA) para o administrador.
- Assinatura digital do instalador do agente.
- Registro de leituras e do conteúdo de arquivos: apenas criação, modificação, renomeação, movimentação e exclusão são registradas.

2.7 Escopo do MVP

Para a entrega do projeto, os requisitos foram divididos em dois grupos. O MVP reúne o mínimo necessário para o produto ser útil e seguro num ambiente real; os requisitos complementares agregam valor, mas não condicionam a entrega.

- **MVP:** RF01 a RF07, RF09, RF11, RF13, RF14, RF16, RF20, RF21, RF22 e RF23. Cobre coleta e envio de métricas e eventos, detecção de inativos, alertas, dashboard, autenticação de administrador e agentes, revogação e retenção.
- **Complementares:** RF08, RF10, RF12, RF15, RF17, RF18 e RF19. São histórico em gráficos e alertas, configuração remota de diretórios e intervalo, usuário responsável pelo evento e atualização automática do agente.

3. Fluxos e Comportamento do Sistema

3.1 Fluxo de Coleta e Envio de Métricas

Descrição

O agente é inicializado no endpoint, lê o arquivo de configuração local (intervalo de coleta, endereço do servidor, token de autenticação e diretórios monitorados), registra o endpoint no servidor e inicia o ciclo contínuo de coleta e envio de dados. Conforme figura 02.

Passo a passo

- Agente é iniciado no endpoint (manualmente ou como serviço do sistema operacional)
- Agente lê o arquivo de configuração local (config.json / config.ini)
- Agente envia requisição de registro ao servidor central com hostname, IP e metadados do sistema
- Servidor valida o token de autenticação e registra ou atualiza o endpoint no banco de dados
- Agente inicia o loop de coleta: aguarda o intervalo configurado (padrão: 60 segundos)
- Agente coleta métricas: percentual de CPU, memória usada/total, espaço em disco usado/total por partição, bytes enviados/recebidos por interface de rede
- Agente serializa as métricas em JSON e envia ao servidor via HTTP POST
- Servidor recebe, valida e persiste os dados. Retorna 200 OK ou código de erro
- Agente registra o resultado e aguarda o próximo ciclo

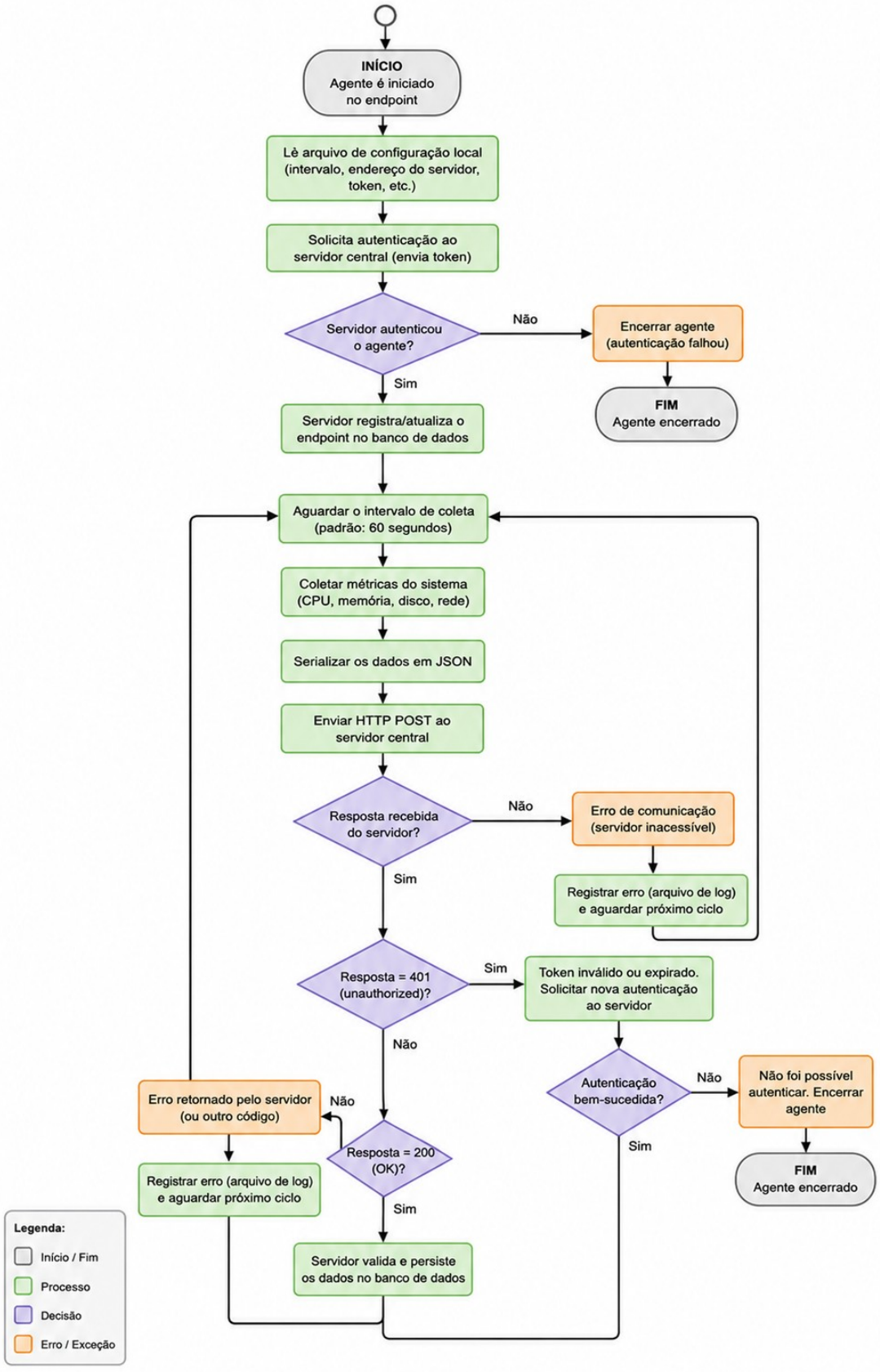


Figura 02: Fluxo de coleta e envio de métricas.

3.2 Fluxo de Detecção de Eventos em Sistema de Arquivos

Descrição

Em paralelo ao ciclo de coleta de métricas, o agente mantém um processo de observação contínua dos diretórios configurados. Ao detectar qualquer alteração (criação, modificação ou exclusão de arquivo), o agente imediatamente envia o evento ao servidor, independente do ciclo de coleta. Conforme figura 03.

Passo a passo

- Agente inicializa o módulo de observação de diretórios com a lista de caminhos configurados
- Sistema operacional notifica o agente sobre alteração no sistema de arquivos (via watchdog / inotify / ReadDirectoryChangesW)
- Agente captura o tipo de evento (criação / modificação / exclusão), caminho do arquivo e timestamp
- No agente instalado como serviço no Windows, o evento aguarda cerca de 3 segundos para ser associado ao registro de auditoria do Windows (log de Segurança) do mesmo arquivo, que informa o usuário que fez a ação, o programa usado e, em acessos pela rede, o IP de origem. Sem esse registro, é usado o usuário com sessão aberta.
- Agente envia o evento ao servidor central via HTTP POST com autenticação
- Servidor valida e persiste o evento, associando-o ao endpoint de origem
- Evento fica disponível para consulta no dashboard via API REST

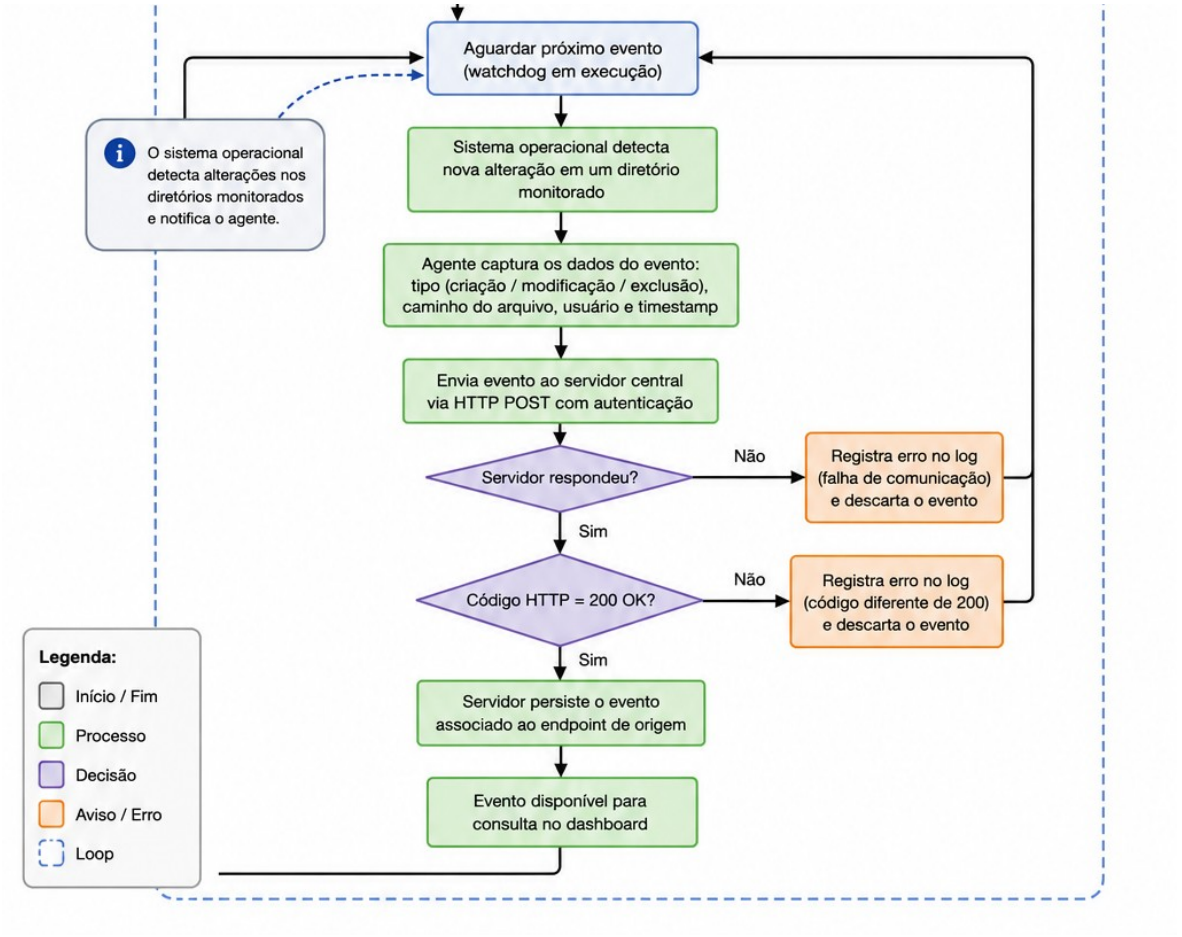


Figura 03: Fluxo de Detecção de Eventos em Sistema de Arquivos.

3.3 Fluxo de Detecção de Endpoint Inativo

Descrição

O servidor central mantém o registro do timestamp da última coleta recebida de cada endpoint. Um job periódico verifica se algum endpoint não enviou dados dentro do tempo esperado, alterando seu status para offline no banco de dados. Conforme figura 04.

Passo a passo

- Job periódico no servidor executa a cada intervalo de verificação (ex.: a cada 2 minutos)
- Para cada endpoint registrado, verifica o timestamp da última coleta recebida
- Se o tempo decorrido desde a última coleta for superior a 3x o intervalo de coleta configurado, o endpoint é marcado como offline
- Dashboard consulta o status atualizado via API e exibe o indicador visual correspondente (offline / alerta)

- Quando o agente voltar a enviar dados, o servidor atualiza automaticamente o status para online

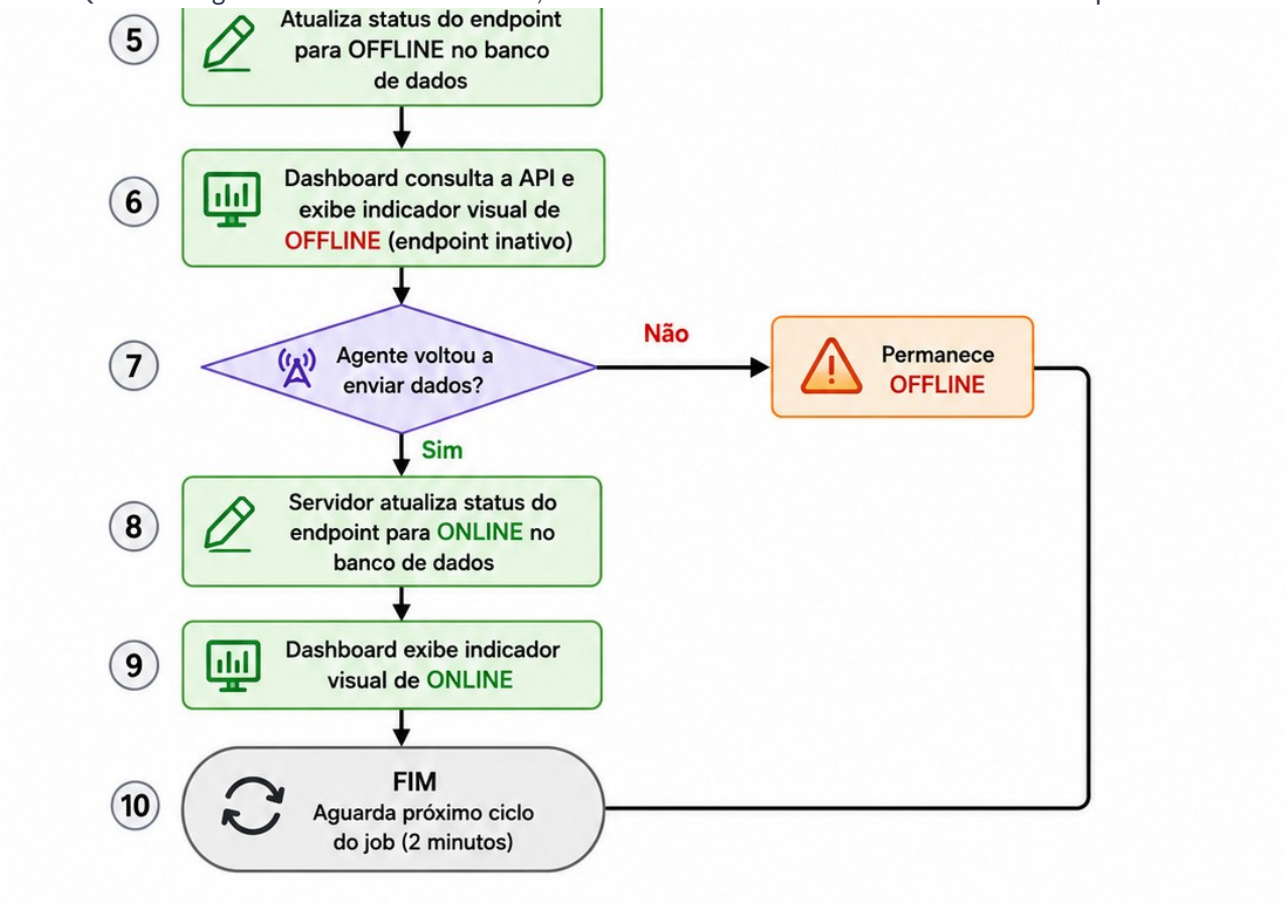


Figura 04: Fluxo de Detecção de Endpoint Inativo.

3.4 Fluxo de Visualização no Dashboard

Descrição

O administrador acessa o dashboard web pelo navegador. A interface consulta periodicamente a API REST do servidor central e atualiza a exibição com os dados mais recentes dos endpoints. Conforme figura 05.

Passo a passo

- Administrador acessa o dashboard via navegador (ex.: http://servidor:3000)
- Dashboard realiza requisição GET à API REST para obter a lista de endpoints e seus status

- Administrador visualiza o painel com cards de cada endpoint indicando: nome, status, última atualização e métricas resumidas
- Ao selecionar um endpoint, o dashboard realiza nova requisição para obter as métricas detalhadas e o histórico
- Dashboard renderiza gráficos de série temporal (CPU, memória, disco) com os dados históricos retornados
- Administrador pode navegar para a aba de Eventos para visualizar os registros de sistema de arquivos do endpoint selecionado
- Dashboard atualiza automaticamente os dados a cada 30 segundos sem necessidade de recarregar a página

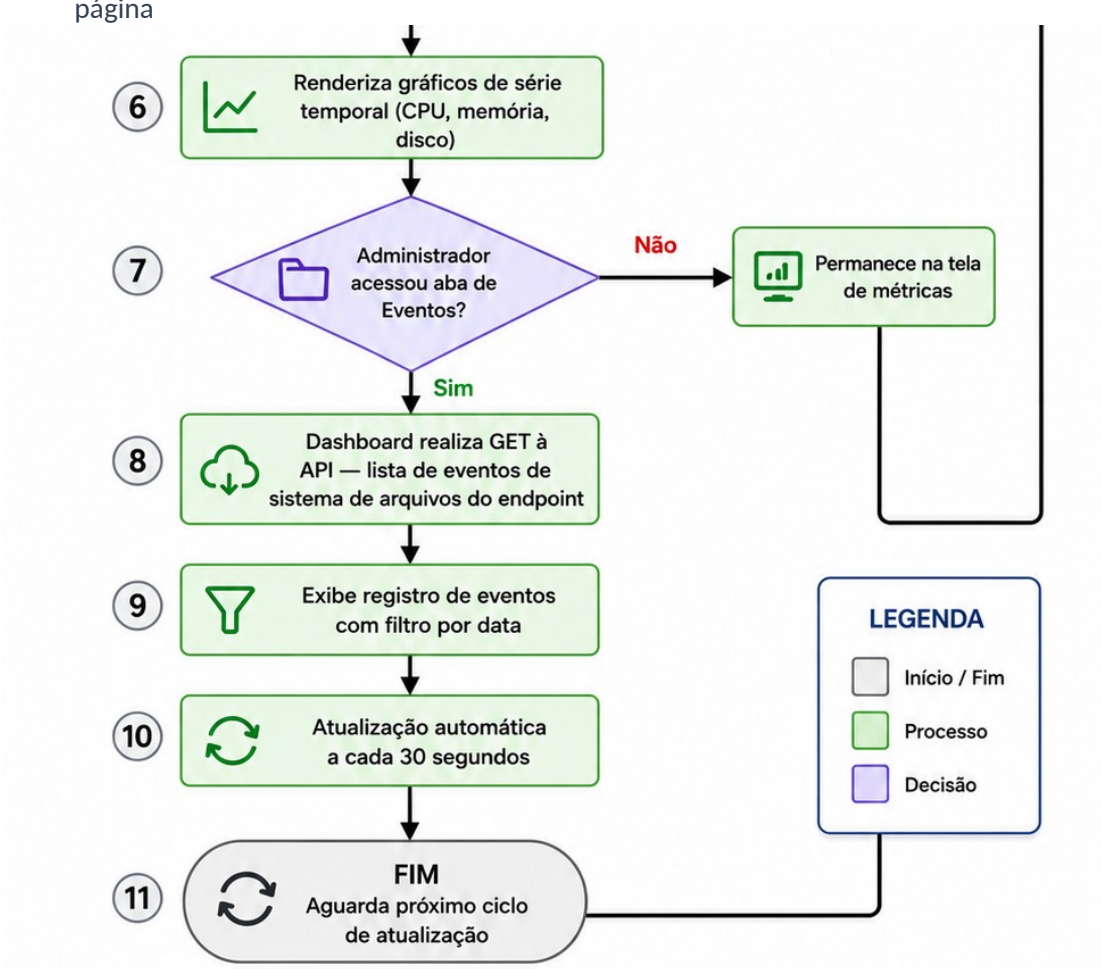


Figura 05: Fluxo de Visualização no Dashboard.

4.1 Fluxo de Navegação

O fluxo principal de navegação do administrador no dashboard segue a estrutura:

Dashboard Geral → **Detalhe do** → **Métricas em** → **Histórico / Eventos**

Endpoint Tempo Real

4.2 Wireframes e Mockups das Telas

Tela 1 — Dashboard Geral (Lista de Endpoints)

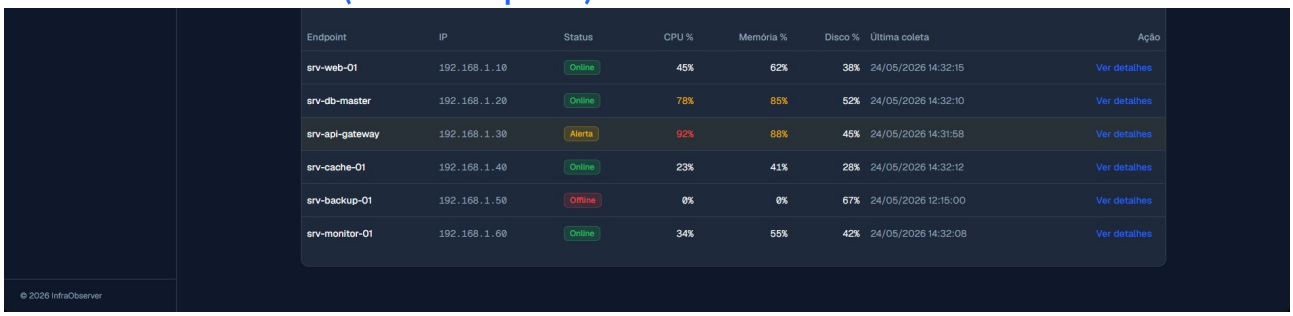


Figura 06: Mockup da tela de Dashboard Geral.

Tela 2 — Detalhe do Endpoint (Métricas e Gráficos)



Figura 07: Mockup da tela de detalhes do endpoint.

Tela 3 — Registro de Eventos de Sistema de Arquivos

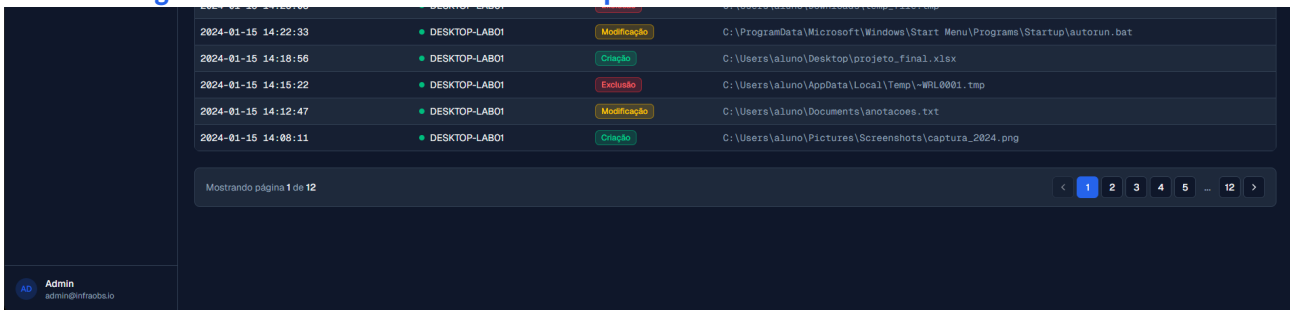


Figura 08: Mockup da tela eventos do sistema de arquivos.

Tela 4 — Página de Alertas

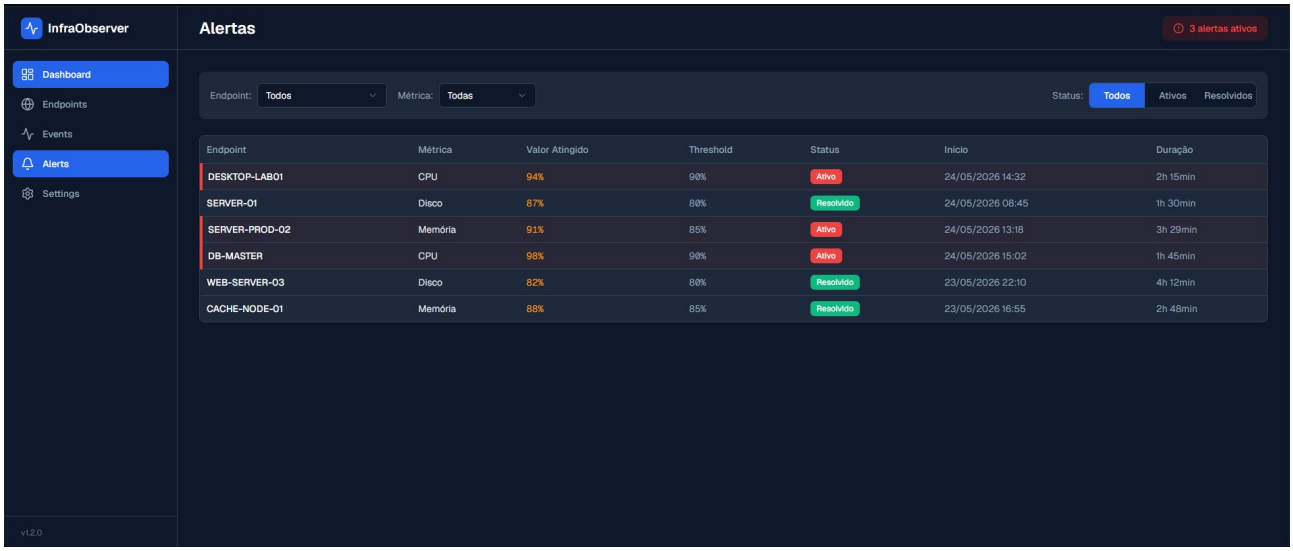


Figura 09: Mockup da tela de alertas.

Tela 5 — Lista de Endpoints (Gerenciamento)



Figura 10: Mockup da tela de lista de endpoints.

Tela 6 — Configurações do Sistema

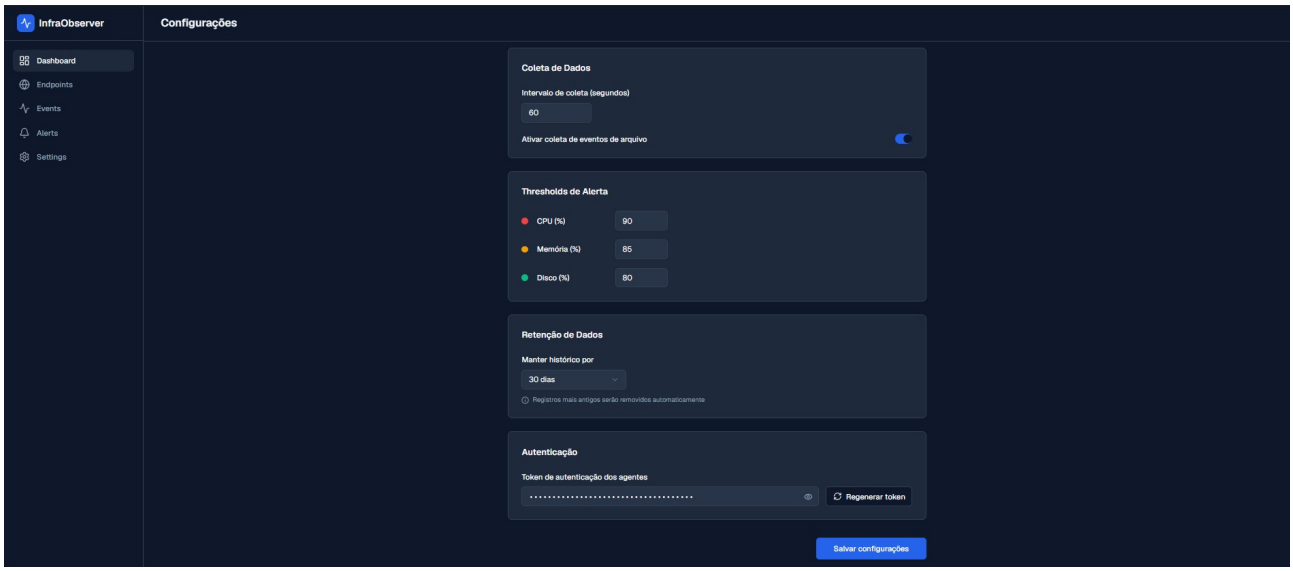


Figura 11: Mockup da tela de configurações do sistema.

Tela 7 — Agente: Tela de Status (instalado no endpoint)

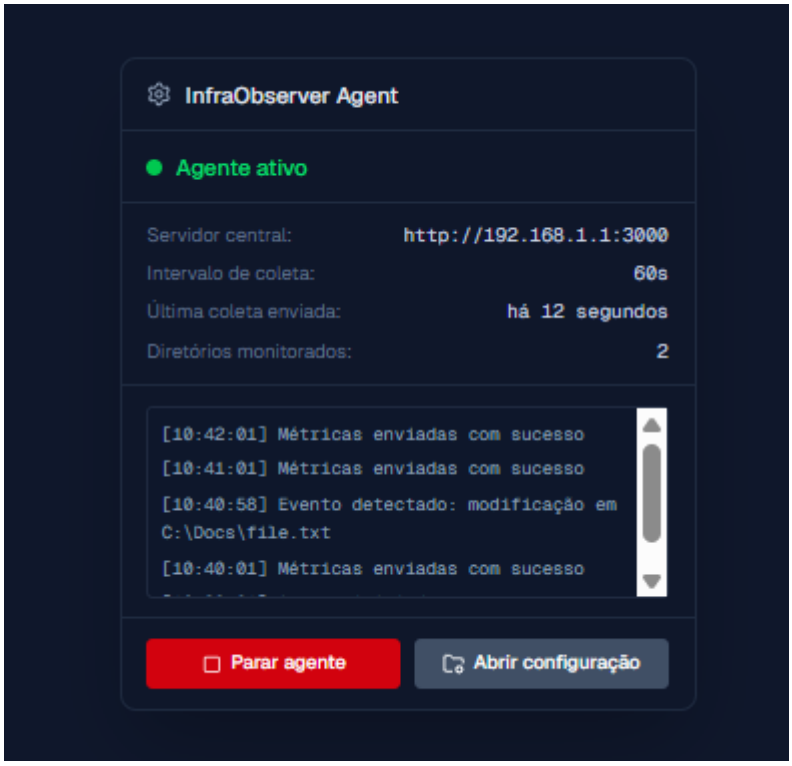


Figura 12: Mockup da tela de status do agente.

5. Arquitetura do Sistema

5.1 Diagrama C4

Nível 1 — Diagrama de Contexto

O InfraObserver é composto por três elementos principais no contexto do sistema: os endpoints monitorados (com o agente instalado), o servidor central (backend da aplicação) e o administrador que acessa o dashboard. Os endpoints se comunicam com o servidor via HTTP. O administrador acessa o dashboard via navegador web. Conforme figura 13.

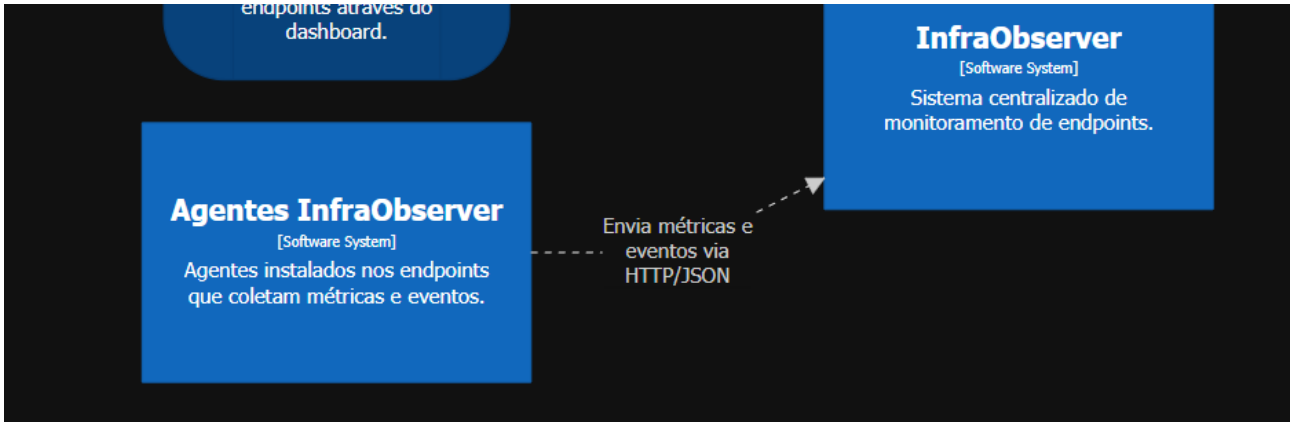


Figura 13: Diagrama de contexto.

Nível 2 — Diagrama de Containers

O sistema é dividido em três containers principais: o Agente (processo Python instalado nos endpoints), o Servidor Central (API REST + banco de dados) e o Dashboard Web (interface frontend). A comunicação entre Agente e Servidor é feita via HTTP/JSON. O Dashboard consome a API REST do Servidor. Conforme figura 14.

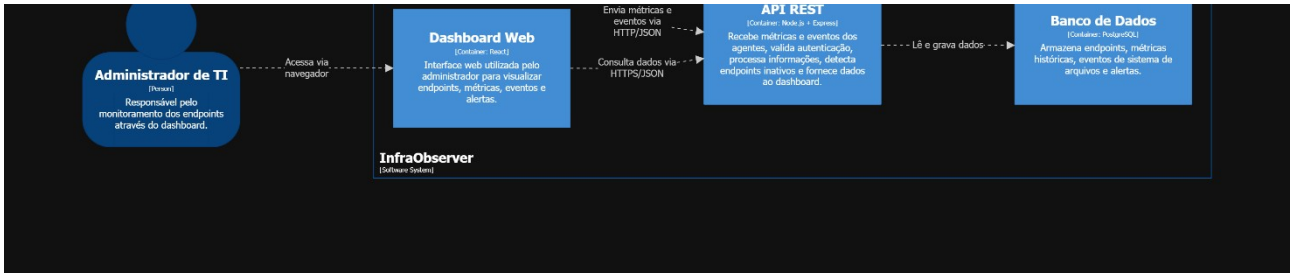


Figura 14: Diagrama de containers.

Nível 3 — Diagrama de Componentes (Servidor Central)

O Servidor Central é organizado em camadas, seguindo a separação de responsabilidades. As requisições dos agentes e do dashboard passam pelo Router, são processadas pelos Controllers, que delegam à camada de Services para a lógica de negócio, e à camada de Repositories para persistência. Conforme figura 15.

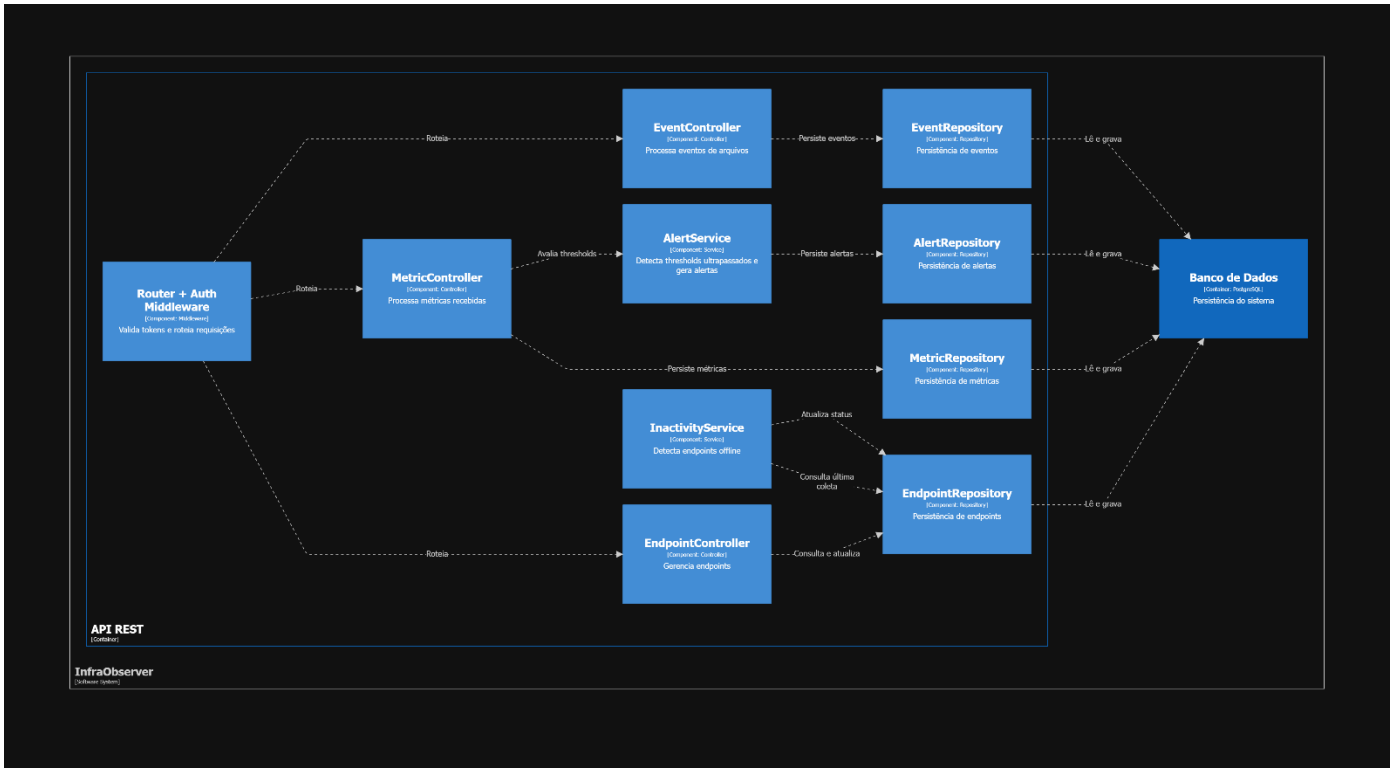


Figura 15: Diagrama de componentes

5.2 Modelo de Dados

O banco de dados é composto por quatro entidades principais. O diagrama Entidade-Relacionamento conceitual é apresentado abaixo na figura 16:

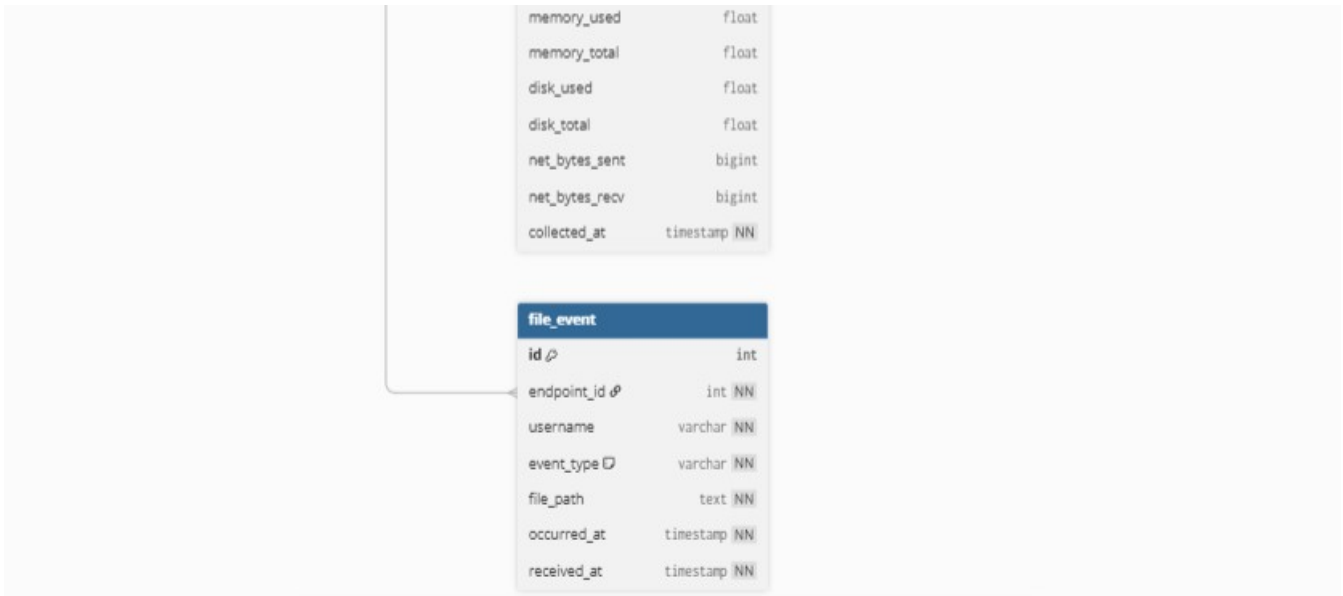


Figura 16: Modelo de dados.

5.3 Principais Componentes

Os principais componentes que compõem a arquitetura do InfraObserver são apresentados na Tabela 08. Cada componente possui responsabilidades específicas e tecnologias definidas, contribuindo para a separação de responsabilidades e a organização da solução.

Componente	Tecnologia	Responsabilidade
Agente (Agent)	Python	Processo leve instalado em cada endpoint. Responsável por coletar métricas via psutil, detectar eventos de arquivo via watchdog e enviar os dados ao servidor via HTTP/JSON. Configurável por arquivo local. Executado como serviço do sistema operacional e configurado por arquivo local.
API REST (Backend)	Node.js + Express	Servidor central responsável por autenticar agentes, receber métricas e eventos, processar regras de negócio, detectar endpoints inativos e disponibilizar dados para o dashboard.
Service Layer	Node.js + TypeScript	Camada responsável pelas regras de negócio do sistema. Processa métricas, gera alertas, detecta endpoints inativos e coordena a comunicação entre controllers e repositories.
Repository Layer	Node.js + Prisma ORM	Camada de persistência responsável por abstrair o acesso ao banco de dados e executar operações de leitura e escrita das entidades do sistema.
Banco de Dados	PostgreSQL	Responsável pelo armazenamento persistente dos endpoints, métricas históricas, eventos de sistema de arquivos, alertas e configurações dos agentes.
Dashboard Web	React	Interface web para o administrador. Consome a API REST via polling periódico. Exibe cards de status dos endpoints, gráficos de série temporal e listagem de eventos de arquivo.
Configuração dos Agentes	PostgreSQL	Responsável por armazenar tokens de autenticação, versão do agente, diretórios monitorados e intervalo de coleta.

Tabela 08: Principais componentes da arquitetura do sistema InfraObserver.

5.4 Stack Tecnológica

A stack tecnológica definida para o desenvolvimento do InfraObserver é apresentada na Tabela 09. As tecnologias foram selecionadas considerando critérios de desempenho, compatibilidade, simplicidade de implantação e aderência aos requisitos do projeto.

Tecnologia	Camada	Justificativa
Python 3.11+	Agente	Linguagem amplamente suportada em Windows e Linux. Biblioteca psutil oferece API unificada para coleta de métricas do sistema operacional. watchdog para monitoramento de sistema de arquivos.
psutil	Agente	Biblioteca Python multiplataforma para coleta de métricas de sistema (CPU, memória, disco, rede) com API simples e sem necessidade de permissões elevadas para uso básico.
watchdog	Agente	Biblioteca Python para monitoramento de eventos em sistema de arquivos. Suporte nativo a inotify (Linux) e ReadDirectoryChangesW (Windows).
Node.js + Express	Servidor Central	Plataforma responsável pela API REST do sistema, recebendo métricas e eventos dos agentes, processando regras de negócio e fornecendo dados ao dashboard.
Prisma ORM	Servidor Central	ORM moderno para Node.js que simplifica o acesso ao banco de dados PostgreSQL, oferecendo tipagem forte, migrações e maior produtividade no desenvolvimento.
PostgreSQL	Servidor Central	Sistema gerenciador de banco de dados relacional robusto, utilizado para armazenar endpoints, métricas históricas, eventos de sistema de arquivos, alertas e configurações dos agentes.
Docker	Infraestrutura	Utilizado para padronizar o ambiente de execução do backend, banco de dados e demais serviços, garantindo portabilidade e facilidade de implantação.
React	Dashboard Web	Biblioteca JavaScript moderna para construção de interfaces reativas, permitindo atualização dinâmica de métricas, eventos e alertas em tempo quase real.
Recharts / Chart.js	Dashboard Web	Biblioteca utilizada para renderização de gráficos de séries temporais das métricas coletadas, permitindo visualização histórica e em tempo quase real.
GitHub Actions	DevOps	Automatiza processos de integração contínua, execução de testes e validação da aplicação a cada alteração enviada ao repositório.

Tabela 09: Principais componentes da arquitetura do sistema InfraObserver.

6. Segurança e Privacidade

O InfraObserver foi projetado considerando requisitos básicos de segurança para garantir a integridade dos dados coletados e restringir o envio de informações apenas a agentes autorizados.

6.1 Autenticação e Controle de Acesso

O sistema tem três tipos de acesso, cada um com uma credencial e um alcance diferentes. Nenhum segredo é armazenado em texto puro e nenhum é registrado em log. A Tabela 10 resume como cada ator se autentica e o que pode fazer.

Ator	Como se autentica	Proteções
Administrador	E-mail e senha (mínimo de 12 caracteres). Após o login, recebe uma sessão JWT em cookie, válida por 8 horas.	Senha guardada só como hash bcrypt (custo 12). Cookie httpOnly, SameSite=Strict e Secure. Bloqueio após 5 tentativas erradas por IP em 15 minutos.
Instalador do agente	Token de instalação do cliente (ioe_), gerado pelo administrador no dashboard e usado por todos os computadores do cliente. Só serve para obter a credencial do computador.	256 bits aleatórios, guardados como hash SHA-256 (e cifrados com AES-256-GCM para poderem ser copiados de novo). Validade opcional e revogação. Bloqueio após 20 tentativas erradas por IP em 15 minutos.
Agente	Credencial própria do computador (ioa_), recebida na primeira conexão e enviada em cada requisição.	Guardada no servidor só como hash. Vale apenas para o próprio endpoint. Pode ser revogada individualmente. No computador, fica em arquivo acessível só a administradores e ao sistema.

Tabela 10: Autenticação e controle de acesso por ator.

Medidas complementares

- Comunicação por HTTPS entre agentes, dashboard e servidor.
- Validação de toda entrada da API por esquemas (Zod) e consultas parametrizadas ao banco (Prisma).
- Proteção contra CSRF e XSS: cookie SameSite=Strict, CORS restrito às origens liberadas e cabeçalhos de segurança (helmet).
- Logs sem senhas, tokens ou cabeçalhos de autorização: apenas identificadores e o prefixo do token.
- Banco de dados e métricas internas sem acesso público, apenas pela rede interna dos containers.
- Boas práticas do OWASP Top 10 e análise estática contínua (SonarQube) no pipeline de integração.

6.2 Privacidade e LGPD

O InfraObserver trata dados pessoais de colaboradores da organização que o utiliza, principalmente nos eventos de sistema de arquivos. Esta seção descreve os dados tratados, a finalidade, a base legal sugerida, a retenção e as medidas de proteção, com base na Lei nº 13.709/2018 (LGPD).

Papéis no tratamento

A organização que implanta o sistema é a controladora dos dados (art. 5º, VI): define as pastas monitoradas, informa os colaboradores e responde aos titulares. Quando o servidor é hospedado por terceiro, este atua como operador (art. 5º, VII) e deve haver contrato definindo suas obrigações (art. 39). Em serventias extrajudiciais, aplicam-se também as regras do Provimento CNJ nº 134/2022, hoje consolidadas no Código Nacional de Normas (Provimento CNJ nº 149/2023).

Dados coletados

A Tabela 11 lista todos os dados coletados, a finalidade de cada um e se é dado pessoal.

Dado	Finalidade	Dado pessoal?
Nome do computador, sistema operacional e versão do agente	Identificar o equipamento e controlar atualizações	Não, em regra
Endereço IP do endpoint	Localizar o equipamento na rede	Indireto: pode identificar o usuário do equipamento
Métricas de CPU, memória, disco e rede, com data e hora	Medir desempenho, gerar alertas e detectar inatividade	Não
Evento de arquivo: tipo da ação, caminho do arquivo e data e hora	Auditoria operacional e suporte	Pode ser, se o nome do arquivo ou da pasta contiver dado pessoal
Usuário que realizou a ação (domínio e nome de usuário)	Saber quem alterou ou apagou um arquivo	Sim
Programa usado e IP de origem do acesso pela rede	Contexto da ação, para investigar incidentes	Sim, associado ao usuário
Conta do administrador: nome, e-mail e hash da senha	Controle de acesso ao dashboard	Sim
Logs do servidor: e-mail em tentativas de login com erro e registro de cópias de token	Segurança e investigação de incidentes	Sim

Tabela 11: Dados coletados pelo InfraObserver.

O sistema não coleta conteúdo de arquivos, leituras de arquivos, telas, teclas digitadas, histórico de navegação, mensagens ou senhas, nem eventos fora das pastas monitoradas. Não há tratamento de dados pessoais sensíveis (art. 5º, II) pelo sistema.

Finalidade e base legal

A finalidade é exclusivamente a segurança da informação e o suporte técnico dos equipamentos da organização (princípio da finalidade, art. 6º, I). A base legal sugerida é o legítimo interesse do controlador (art. 7º, IX), desde que a organização documente o teste de balanceamento recomendado pelo Guia Orientativo da ANPD sobre legítimo interesse (2024) e observe o art. 10: tratar só os dados estritamente necessários e garantir transparência aos titulares. A definição final da base legal cabe ao controlador e ao seu encarregado.

Transparência e minimização

- A organização deve informar os colaboradores, por política interna ou aviso, antes de ativar o monitoramento, indicando quais pastas são monitoradas e para quê.
- Apenas as pastas escolhidas na instalação são monitoradas, e a auditoria do Windows pode ser desativada por computador. Nesse caso, o evento registra apenas o usuário com sessão aberta.
- O dashboard indica, em cada evento, se o usuário veio da auditoria do Windows ou da sessão aberta, evitando atribuir uma ação a alguém sem evidência.

Retenção e eliminação

Métricas, eventos de sistema de arquivos e alertas são mantidos por 60 dias, prazo padrão definido com o ambiente piloto e configurável pela organização. Uma rotina diária no servidor elimina os registros mais antigos (art. 16). Os dados aguardando envio no próprio computador, quando o servidor está fora do ar, ficam no máximo 7 dias. Ao revogar o acesso de um endpoint, seu histórico permanece apenas até o fim do mesmo prazo. Contas de administrador são mantidas enquanto ativas e excluídas quando o acesso deixa de ser necessário.

Segurança e incidentes

As medidas de segurança técnicas (art. 46) estão descritas na seção 6.1. Em caso de incidente, a organização controladora avalia o risco e comunica a ANPD e os titulares quando necessário (art. 48). O sistema apoia a resposta permitindo revogar imediatamente tokens e endpoints e mantendo logs que identificam o acesso sem expor segredos.

Direitos dos titulares e avaliação de impacto

As solicitações dos titulares (art. 18), como acesso ou eliminação, são atendidas pelo encarregado da organização, que pode localizar os eventos de um colaborador pela busca do dashboard. Como o sistema registra ações de colaboradores, recomenda-se que a organização elabore um Relatório de Impacto à Proteção de Dados Pessoais (art. 38) antes da implantação.

7. Planejamento do Projeto

O planejamento do desenvolvimento do InfraObserver é apresentado na Tabela 12. Os marcos estabelecidos definem as principais etapas do projeto, organizadas em ordem cronológica para orientar a implementação e o acompanhamento da evolução da solução.

Marco	Descrição	Prazo
M1	Conclusão da documentação do projeto (RFC, requisitos, diagramas C4, casos de uso, fluxos e modelo de dados) e configuração inicial do repositório GitHub.	Semana 1
M2	Desenvolvimento da estrutura base do backend (Node.js + Express), banco PostgreSQL e arquitetura Controller → Service → Repository.	Semana 2
M3	Desenvolvimento do agente em Python para coleta de métricas de CPU, memória, disco e rede, incluindo comunicação HTTP/JSON com o servidor central.	Semana 3
M4	Implementação do monitoramento de eventos de sistema de arquivos utilizando	Semana 4

	Watchdog e persistência dos eventos no servidor central.	
M5	Desenvolvimento do Dashboard Web em React com listagem de endpoints, status online/offline e visualização de métricas em tempo quase real.	Semana 5
M6	Implementação de gráficos históricos, alertas por threshold e detecção automática de endpoints inativos.	Semana 6
M7	Testes integrados entre agente, backend e dashboard, correções de bugs e otimizações de desempenho.	Semana 7
M8	Configuração de Docker, GitHub Actions (CI/CD), elaboração da documentação final e preparação para apresentação do projeto.	Semana 8

Tabela 12: Planejamento do desenvolvimento do projeto InfraObserver.

Observação: O desenvolvimento seguirá uma abordagem incremental baseada em Kanban, permitindo a validação contínua das funcionalidades implementadas e a entrega progressiva do MVP do InfraObserver.

7.1 Validação do Produto

A validação combina um piloto em ambiente real com testes que medem as metas da seção 1.6. O ambiente piloto é um cartório de registro de imóveis com 45 estações Windows em domínio, mesmo perfil da Persona 1. A Tabela 13 mostra o que é validado, como e o critério de aceite.

O que validar	Como	Critério de aceite
Utilidade em ambiente real	Piloto com o agente instalado em computadores do ambiente piloto, acompanhando alertas e eventos de arquivo no dia a dia	Agentes online sem intervenção manual; eventos atribuídos ao usuário do domínio; nenhum impacto percebido pelos usuários
Usabilidade do dashboard	Questionário SUS (System Usability Scale) aplicado a profissionais de TI após tarefas guiadas no dashboard	Pontuação SUS média de pelo menos 68
Desempenho da API	Teste de carga (k6) simulando 50 endpoints enviando métricas a cada 60 segundos e o dashboard consultando a API	95% das requisições em menos de 300 ms, sem erros
Consumo do agente	Medição do uso de CPU do serviço do agente durante 24 horas num computador do piloto	Média inferior a 2% de CPU
Detecção de inativos e de eventos	Desligar um computador monitorado; criar, alterar e apagar arquivos numa pasta monitorada	OFFLINE após 3 ciclos; eventos no dashboard em até 10 segundos
Instalação, auditoria e remoção	Teste automatizado no pipeline, num Windows real, a cada alteração do agente	Serviço ativo, auditoria ligada e desfeita na desinstalação

O que validar	Como	Critério de aceite
Segurança e retenção	Testes automatizados de autenticação, revogação e limpeza de dados antigos	Credenciais revogadas recusadas; registros com mais de 60 dias removidos

Tabela 13: Plano de validação do InfraObserver.

8. Referências

C4 Model
BROWN, Simon. The C4 Model for Visualising Software Architecture. Disponível em: https://c4model.com. Acesso em: 17 jun. 2026.

Structurizr
STRUCTURIZR. Structurizr DSL Documentation. Disponível em: https://docs.structurizr.com/dsl. Acesso em: 17 jun. 2026.

Node.js
NODE.JS FOUNDATION. Node.js Documentation. Disponível em: https://nodejs.org/docs. Acesso em: 17 jun. 2026.

Express
EXPRESS.JS. Express Documentation. Disponível em: https://expressjs.com. Acesso em: 17 jun. 2026.

Python
PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: https://docs.python.org/3. Acesso em: 17 jun. 2026.

psutil
GIAMPAOLO RODOLÀ. psutil Documentation. Disponível em: https://psutil.readthedocs.io. Acesso em: 17 jun. 2026.

Watchdog
WATCHDOG PROJECT. Watchdog Documentation. Disponível em: https://python-watchdog.readthedocs.io. Acesso em: 17 jun. 2026.

React
META. React Documentation. Disponível em: https://react.dev. Acesso em: 17 jun. 2026.

PostgreSQL
POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs. Acesso em: 17 jun. 2026.

Docker
DOCKER INC. Docker Documentation. Disponível em: https://docs.docker.com. Acesso em: 17 jun. 2026.

OWASP

OWASP FOUNDATION. OWASP Top 10 Web Application Security Risks. Disponível em: https://owasp.org/www-project-top-ten. Acesso em: 17 jun. 2026.

FIELDING, Roy Thomas. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral Dissertation, University of California, Irvine, 2000.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

LGPD
BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018.
ANPD
AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. Guia Orientativo: Hipóteses Legais de Tratamento de Dados Pessoais — Legítimo Interesse. Versão 1.0. Brasília: ANPD, fev. 2024.
CNJ
CONSELHO NACIONAL DE JUSTIÇA. Provimento nº 134, de 24 de agosto de 2022. Estabelece medidas a serem adotadas pelas serventias extrajudiciais para adequação à LGPD. Brasília: CNJ, 2022.
CNJ
CONSELHO NACIONAL DE JUSTIÇA. Provimento nº 149, de 30 de agosto de 2023. Institui o Código Nacional de Normas da Corregedoria Nacional de Justiça — Foro Extrajudicial. Brasília: CNJ, 2023.
SUS
BROOKE, John. SUS: A "quick and dirty" usability scale. In: JORDAN, P. W. et al. (ed.). Usability Evaluation in Industry. London: Taylor & Francis, 1996.

9. Apêndices

Repositório do Projeto

O código-fonte, documentação técnica e artefatos produzidos durante o desenvolvimento do InfraObserver serão mantidos em repositório GitHub público.

Repositório Oficial:
InfraObserver - GitHub
O repositório armazenará:

- Código-fonte do Agente (Python);
- Código-fonte da API REST (Node.js + Express);
- Código-fonte do Dashboard Web (React);
- Scripts de banco de dados;
- Documentação técnica;
- Diagramas arquiteturais;
- Pipeline de Integração Contínua (CI/CD);
- Histórico de Issues e evolução do projeto.

Histórico de revisões

Versão 1.0 — proposta original avaliada pelo comitê.

Versão 1.1 — ajustes conforme o parecer da avaliação (validação e LGPD; reduzir metas do MVP; detalhar autenticação, retenção e dados coletados):

- Metas de desempenho ajustadas ao ambiente piloto (seção 1.6) e definição do escopo do MVP (seção 2.7).
- Autenticação detalhada por ator, com token de instalação e credencial por endpoint (seção 6.1 e requisitos RF14, RF21, RF22, RNF15, RN04, RN11 e RN12).
- Seção de LGPD reescrita: papéis, dados coletados, finalidade, base legal, retenção de 60 dias, incidentes e direitos dos titulares (seção 6.2, RF23, RNF08 e RN06).
- Plano de validação com piloto, teste de usabilidade e testes de desempenho (seção 7.1).

10. Parecer do Comitê de Avaliação

(A ser preenchido pelos professores)

Avaliador 1: _____
Status: [ ] Aprovado [ ] Ajustar
Observações:

Avaliador 2: _____
Status: [ ] Aprovado [ ] Ajustar
Observações:
